



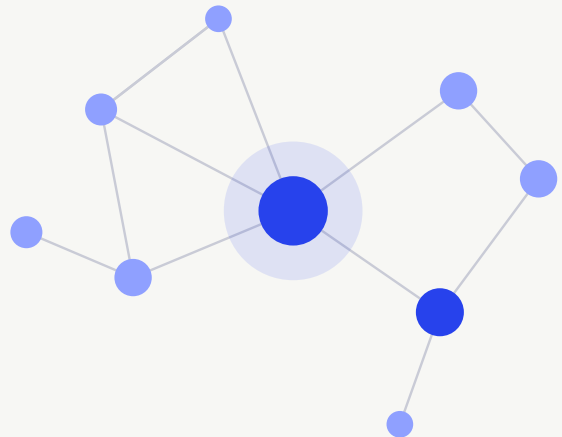
Plexus

REACT JS CASE STUDY · PROJECT DOCUMENTATION

Plexus.

Network Centrality Lab

A browser-native graph analytics product that finds what holds every network together.



SUBMITTED BY

Saish Bhujbal

ROLL NUMBER

150096725063

COHORT

Jeff Bezos

ITM Skills University · B.Tech CSE 2025–29 · Semester II

plexus-eight.vercel.app

live demo

PROBLEM STATEMENT

Finding the center of gravity.

Modern systems are dense relational webs — a social network of users, a service mesh in production, a transaction ledger between accounts, a supply chain across vendors. Looking at the raw edge list reveals nothing. The interesting question is structural: among hundreds of interconnected nodes, which ones actually hold the network together? Which ones bridge separate communities? Which ones, if they disappeared, would split the system into disconnected islands?

Answering these questions requires running graph-theoretic centrality algorithms — degree, closeness, betweenness, eigenvector — across the entire topology. The existing tools fall into two camps: heavyweight desktop software like Gephi and Cytoscape that few analysts have installed, or cloud platforms like Neo4j Aura that require uploading proprietary network data to a third-party server. Neither is ideal when the graph in question is an internal org chart, a sensitive customer interaction map, or a financial transaction trail.

Plexus takes a different architectural stance. Every algorithm — BFS shortest path, Brandes' betweenness, eigenvector power iteration, force-directed layout physics — executes entirely inside the user's browser tab. There is no backend, no upload, no account required, and no data ever leaves the device. The product opens, the graph is computed, and the analyst gets answers in milliseconds.

TECH STACK

Deliberately minimal.

Plexus is built on a small, fully owned stack. The case study requires the centrality algorithms to be implemented from scratch — using a graph library would defeat that purpose — so the dependency footprint is kept intentionally tiny.

Runtime

React 18	Functional components, hooks (useState, useEffect, useRef, useMemo). Used purely as a rendering layer.
Vite	Build tool and dev server. Zero-config, instant HMR, ships a static bundle.
JavaScript	Plain JSX, no TypeScript — keeps the case-study codebase scannable in a viva setting.
SVG	Native browser graphics. Every node is a <circle>, every edge is a <line>. No Canvas API, no WebGL, no charting library.

Algorithms (hand-written, no library)

BFS	Breadth-first traversal used as the primitive for closeness and unweighted shortest path.
Brandes' algorithm	$O(V \cdot E)$ betweenness centrality. Replaces the naive $O(V^3)$ approach so even dense graphs update instantly.
Power iteration	Eigenvector centrality — 100 iterations with normalization per step.
Force simulation	~30 lines of physics: pairwise repulsion (g/d^2), edge springs ($k \cdot dx$), and a gentle center pull. Runs at 60fps in requestAnimationFrame.

State, storage, hosting

State	useState for graph data; useRef for the live simulation alpha and drag target (values that change every frame but don't need React re-renders).
Persistence	window.localStorage. Graphs autosave; the lab restores them on next visit.
Styling	Inline styles for everything component-scoped; a single <style> tag for keyframe animations and pseudo-classes.
Deployment	Vercel. Static output, no server runtime — matches the privacy claim.

COMPONENT ARCHITECTURE

How the parts fit together.

Two layers, cleanly separated: a pure math layer with no React in it, and a React rendering layer that subscribes to its output.

React tree

```
App (top-level router; view = home | pricing | lab)
|
+-- Nav (sticky, frosted, active-link state)
|
+-- Page: Home
|   +-- Hero (title + CTAs)
|   +-- GraphLab <-- live, embedded
|   +-- Feature trio (Reveal cards)
|   +-- Metrics cards (the four lenses)
|   +-- Final CTA
|
+-- Page: Pricing
|   +-- Three plan cards (Explorer / Analyst / Org)
|   +-- Stats strip
|   +-- FAQ accordion
|
+-- Page: Lab
    +-- GraphLab (full size)
        +-- Toolbar (metric chips, mode, presets, I/O)
        +-- SVG Canvas
            |   +-- Edges (animated path overlay)
            |   +-- Nodes (sized by active metric)
            |   +-- Interactions (click, drag, hover, dbl-click)
        +-- Side Panel
            |   +-- Critical Bridges chips
            |   +-- Ranked node table
        +-- Telemetry HUD (counts . density . diameter . ms)
```

Math layer (pure functions)

- **buildAdj(nodes, edges)** `Map<id, Set<id>>` — adjacency list, built once per change.
- **bfsDistances(adj, src)** `Map<id, distance>` — used by closeness and diameter.
- **shortestPath(adj, src, dst)** Ordered array of node ids — drives the orange traced path.
- **computeCentrality(nodes, edges)** `{ deg, clo, bet, eig, adj, density, diameter, ms }` — single call returns everything the UI needs.

Render loop

A single **requestAnimationFrame** loop integrates the force simulation. State that changes every frame — node positions, velocities, simulation alpha — lives in refs to avoid React reconciliation. A trailing **setTick(t => t + 1)** nudges React to repaint only after positions are advanced.

FEATURES

What the product actually does.

Centrality engine

- **Degree centrality** direct connections normalized by $N - 1$.
- **Closeness centrality** reciprocal of total shortest-path distance, via BFS.
- **Betweenness centrality** Brandes' algorithm — every node scored by how often it sits on others' shortest paths.
- **Eigenvector centrality** power iteration over 100 steps with per-step normalization.
- **Live network statistics** density, diameter, and exact compute time in milliseconds.

Interactive canvas

- **Force-directed auto-layout** repulsion between every pair, springs along every edge.
- **Shortest-path tracing** click any two nodes; BFS draws the route in orange with a flowing dash animation.
- **Drag to rearrange** physics reheats on release so the layout settles cleanly.
- **Hover for details** node's score appears beside it; its edges light up cobalt.
- **Dynamic node sizing** radius and colour both scale with the active centrality metric.
- **Top-node halo** the highest-ranked node gets a breathing cobalt glow.

Build mode

- **Click empty canvas** spawns a new node at the cursor.
- **Click node A, then node B** adds an undirected edge between them.
- **Double-click a node** removes it and every edge that touched it.

Data flow

- **Edge-list import** paste a CSV or space-separated list in a modal; the lab parses and lays out the new graph.
- **CSV export** every metric for every node, ready to drop into Excel or a paper.
- **Browser autosave** localStorage holds the current graph; close the tab and come back to it.

Graph presets

- **Social circle** 12 named nodes with dense friendship ties — general demo.
- **Two communities + broker** a single 'Hub' node bridges two cliques — the canonical betweenness example.
- **Star** one core and ten spokes — degree centrality made obvious.
- **Random network** 16 sparsely connected nodes for exploration.

Product surface

- **Three pages** Home, Pricing, and Lab — routed by view state, no external router needed.
- **Sticky frosted nav** active link state, mobile-friendly.
- **Scroll reveals** IntersectionObserver-driven fade-up on every card section.
- **Three-tier pricing** Explorer (free), Analyst (Rs. 499/mo), Org (Rs. 1,999/mo).
- **FAQ accordion** animated expand/collapse with cobalt + icon rotation.
- **Responsive** the lab toolbar wraps on narrow screens; canvas scales fluidly.

PROJECT AT A GLANCE

By the numbers.

4	0	3	~10
centrality metrics implemented from scratch	external graph or visualization libraries	pages — Home, Pricing, Lab	milliseconds typical compute time

SCREENSHOTS

The product, captured live.

Screens captured from the deployed instance at plexus-eight.vercel.app. The graph visible in the landing capture is computed in-browser from the embedded sample dataset.

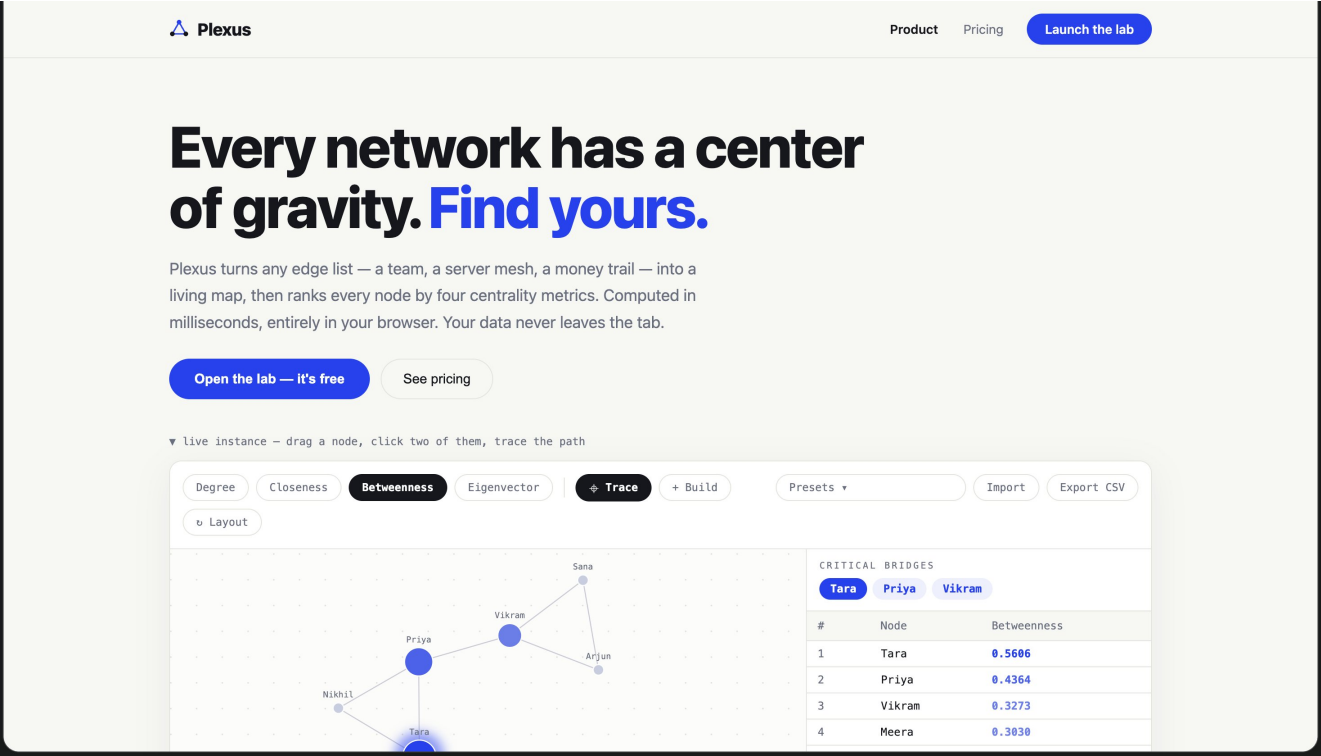


Fig 1 — Landing page hero with the embedded, live graph lab.

SCREENSHOTS

The product, captured live.

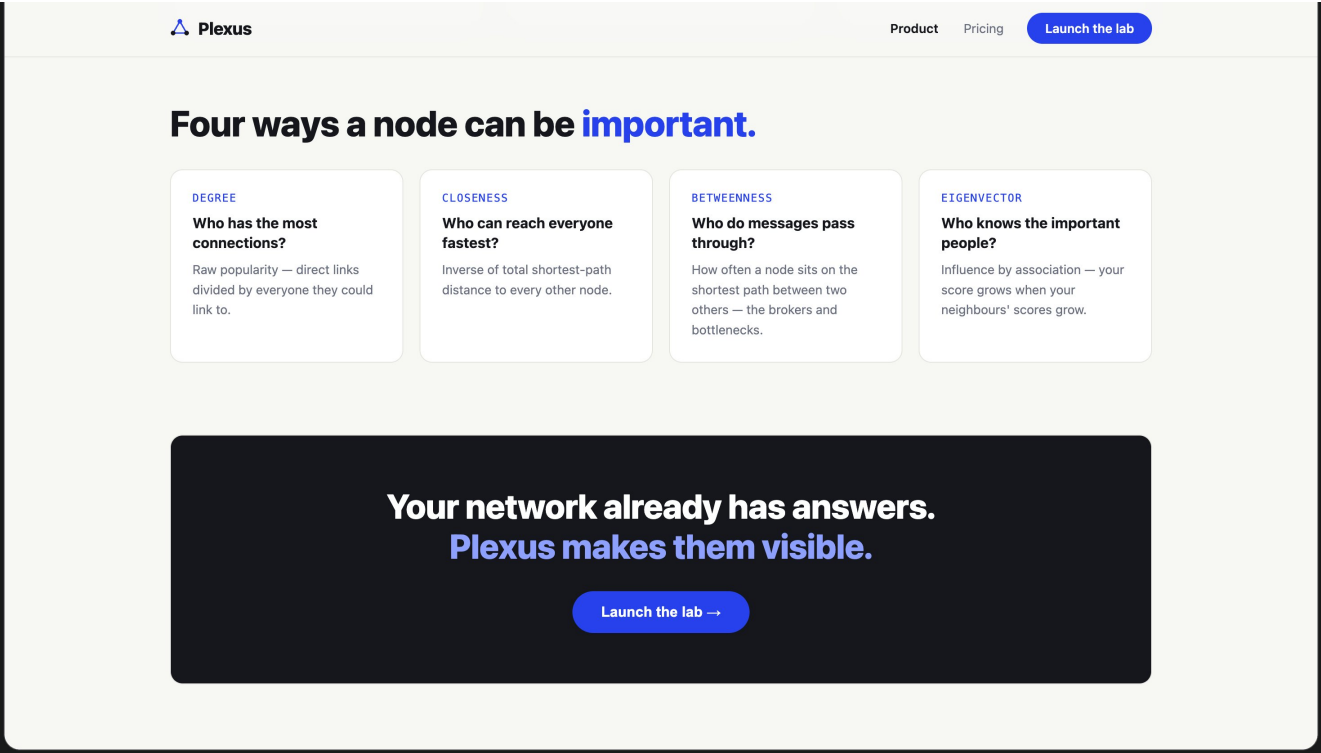


Fig 2 — Metric explainer cards and the final call-to-action.

SCREENSHOTS

The product, captured live.

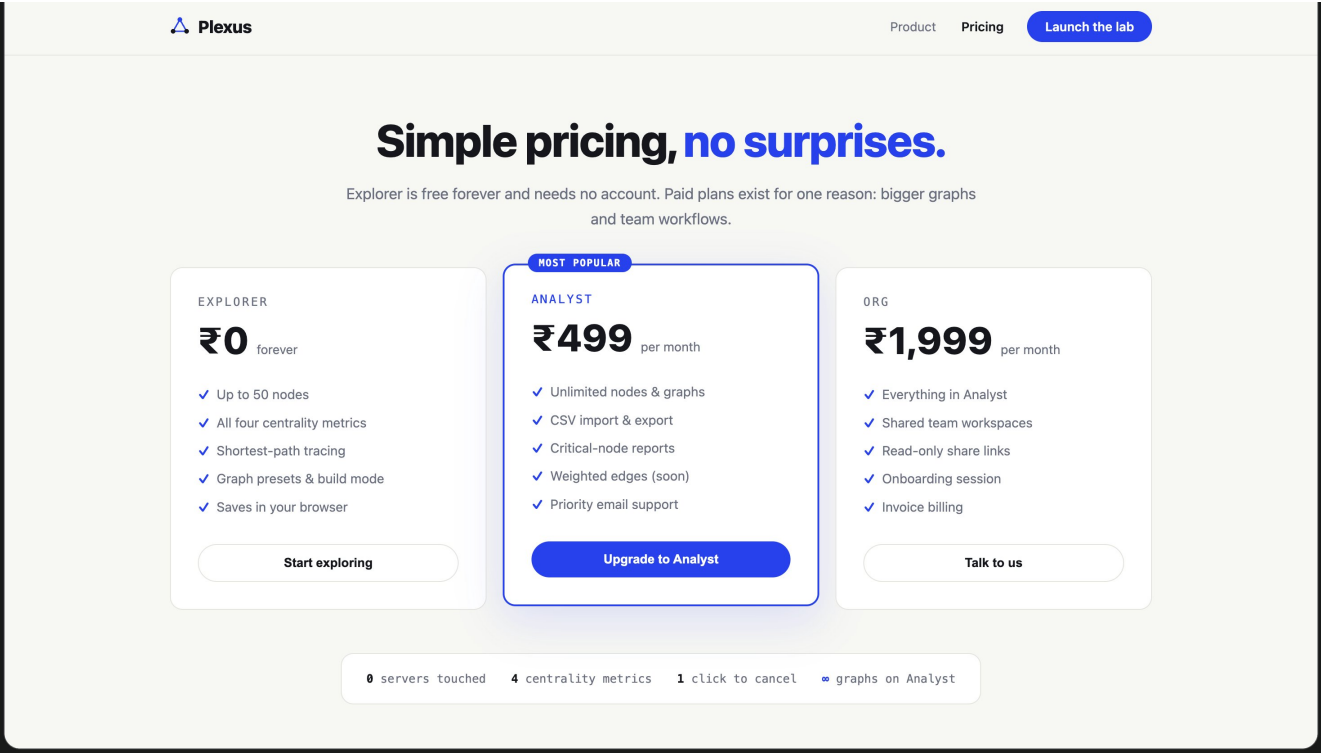


Fig 3 — Dedicated pricing page with the Analyst tier featured.

CONCLUSION

Outcome.

Plexus demonstrates that serious graph analytics can ship as a product rather than as an internal tool. Every centrality formula in the case-study brief — degree, closeness, betweenness, eigenvector — is implemented from scratch, named in the interface, and exposed alongside a live millisecond-precision compute readout. Every traversal (Brandes' algorithm, BFS shortest path) runs entirely client-side, validating the "no backend, no upload" architecture.

On the engineering side, the project shows that React, raw SVG, and a hand-rolled force simulation are enough to deliver a fluid, interactive graph experience without any external visualization library. The deliberate choice to avoid d3 or react-flow makes the case study honest: every algorithm that earns marks is visible in the codebase.

On the product side, the surface — landing, pricing, lab — frames the academic project as a launchable prototype. The free Explorer tier covers the case-study scope; the paid Analyst and Org tiers sketch out a realistic SaaS path for a niche analyst tool. Whether or not it ever monetises, the documentation and live deployment turn a semester assignment into something portfolio-shaped.

In one line: Plexus turns any edge list into a living map of importance — computed where the data already lives.

LINKS

Try it. Read it.

Live demo plexus-eight.vercel.app

Source code github.com/saishbhujbal/plexus

*Submitted by Saish Bhujbal · Roll no. 150096725063 · Cohort Jeff Bezos
ITM Skills University · B.Tech CSE 2025–29 · Semester II*